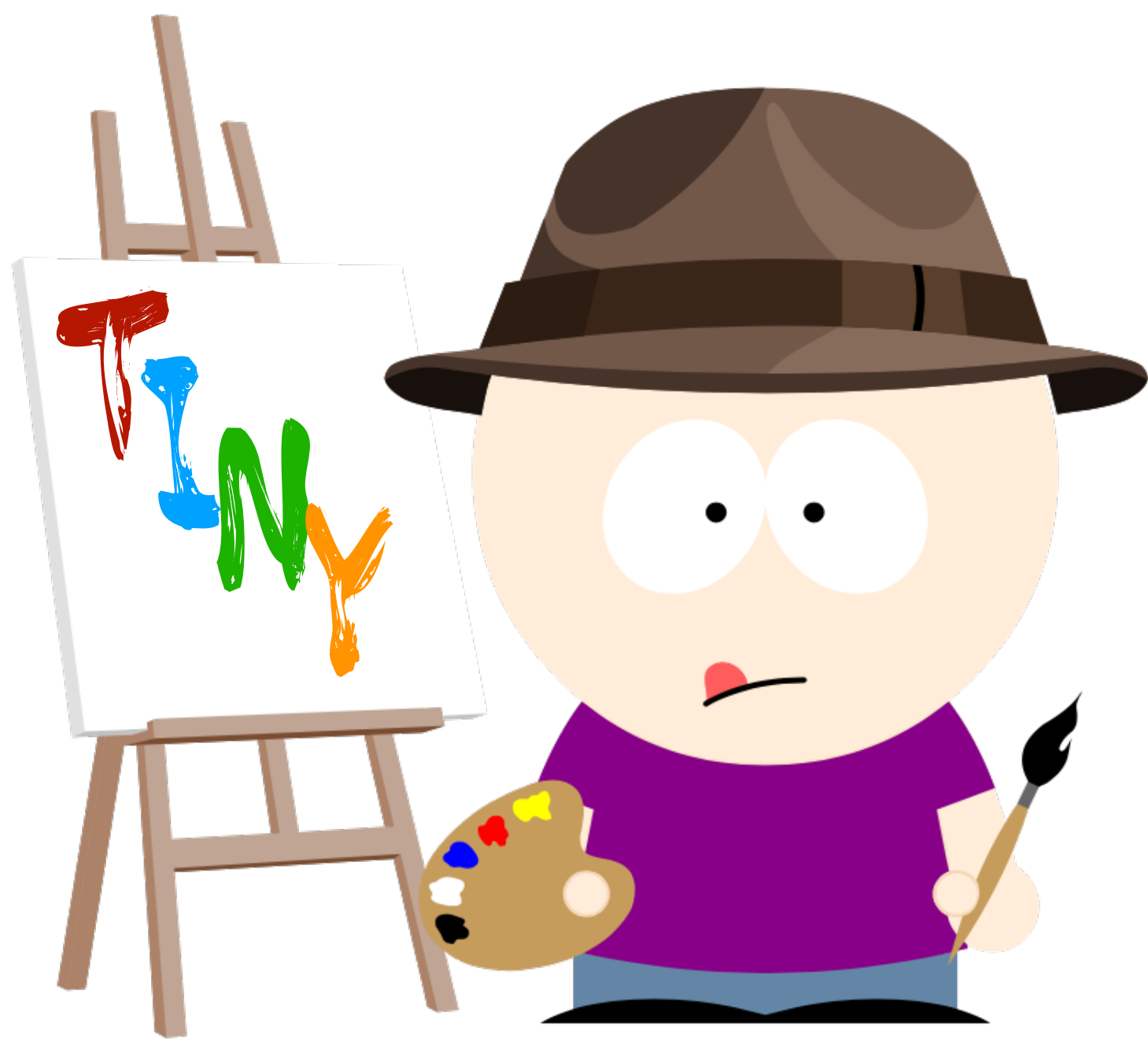




IN-CLASS EXERCISES



SEMANTIC ANALYSIS 1

Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`sem-1-exercises.zip`

3. Open up a terminal and Unzip the file

```
> unzip sem-1-exercises.zip
```

```
> cd sem-1-exercises
```

```
> ls
```

THE TINY 2 LANGUAGE

Tiny 2 Language Extensions

- In this and the following lectures on semantic analysis we will return to the Tiny language. It has been extended in the following ways:

1. Floating point constants (42.67)

2. More arithmetic operators: +, -, *, /

3. Constant declarations:

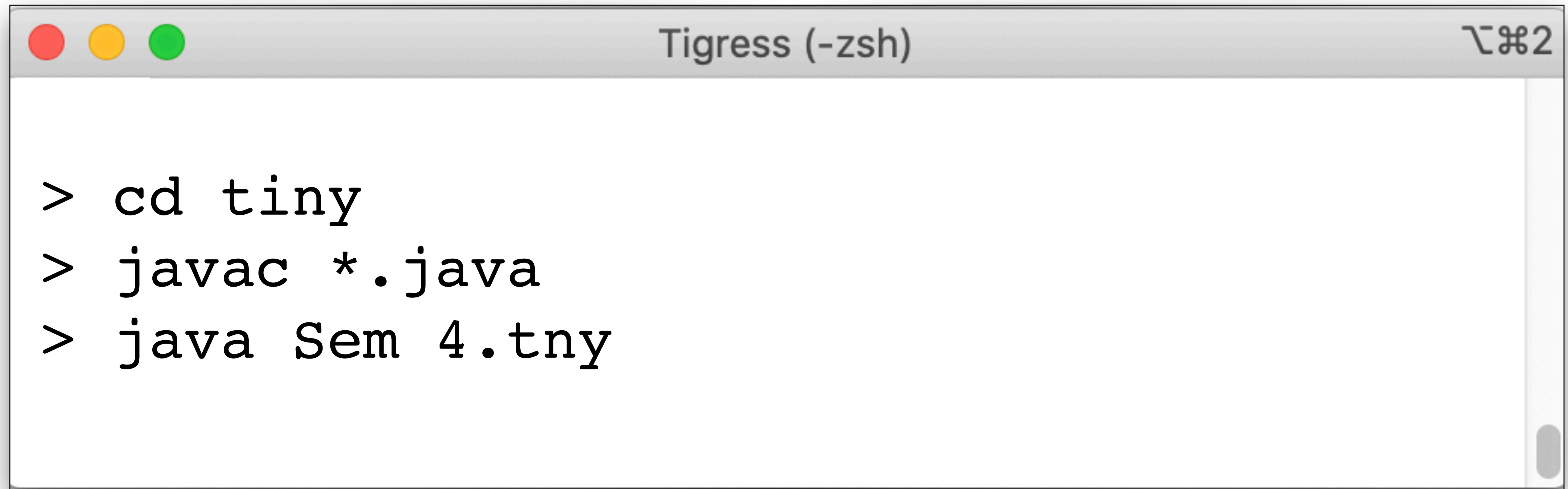
```
BEGIN
    CONST x: int = 4;
    CONST y: int = 5;
    CONST z: int = x * y;
END
```

4. Variable declarations:

```
BEGIN
    VAR x : int;
    PRINT x;
END
```

Getting Started

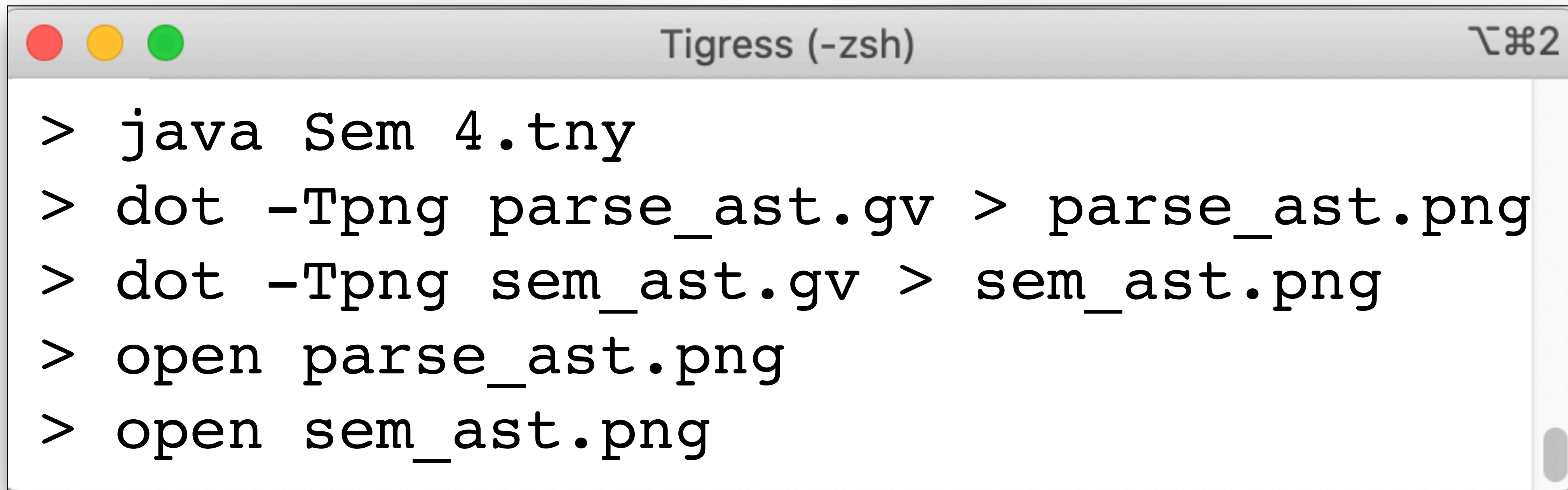
- Here is how you compile the Tiny 2 compiler:

A terminal window titled "Tigress (-zsh)" with standard macOS window controls (red, yellow, green buttons) and a zoom icon in the top right. The terminal contains three lines of commands, each preceded by a prompt character ">".

```
> cd tiny
> javac *.java
> java Sem 4.tny
```

Visualizing the AST

- To be able to debug the semantic analyzer, the Tiny 2 compiler generates visualizations of the AST after parsing, and after semantic analysis:

A terminal window titled "Tigress (-zsh)" with standard macOS window controls (red, yellow, green buttons) and a window icon. The terminal contains five lines of commands, each preceded by a prompt character ">".

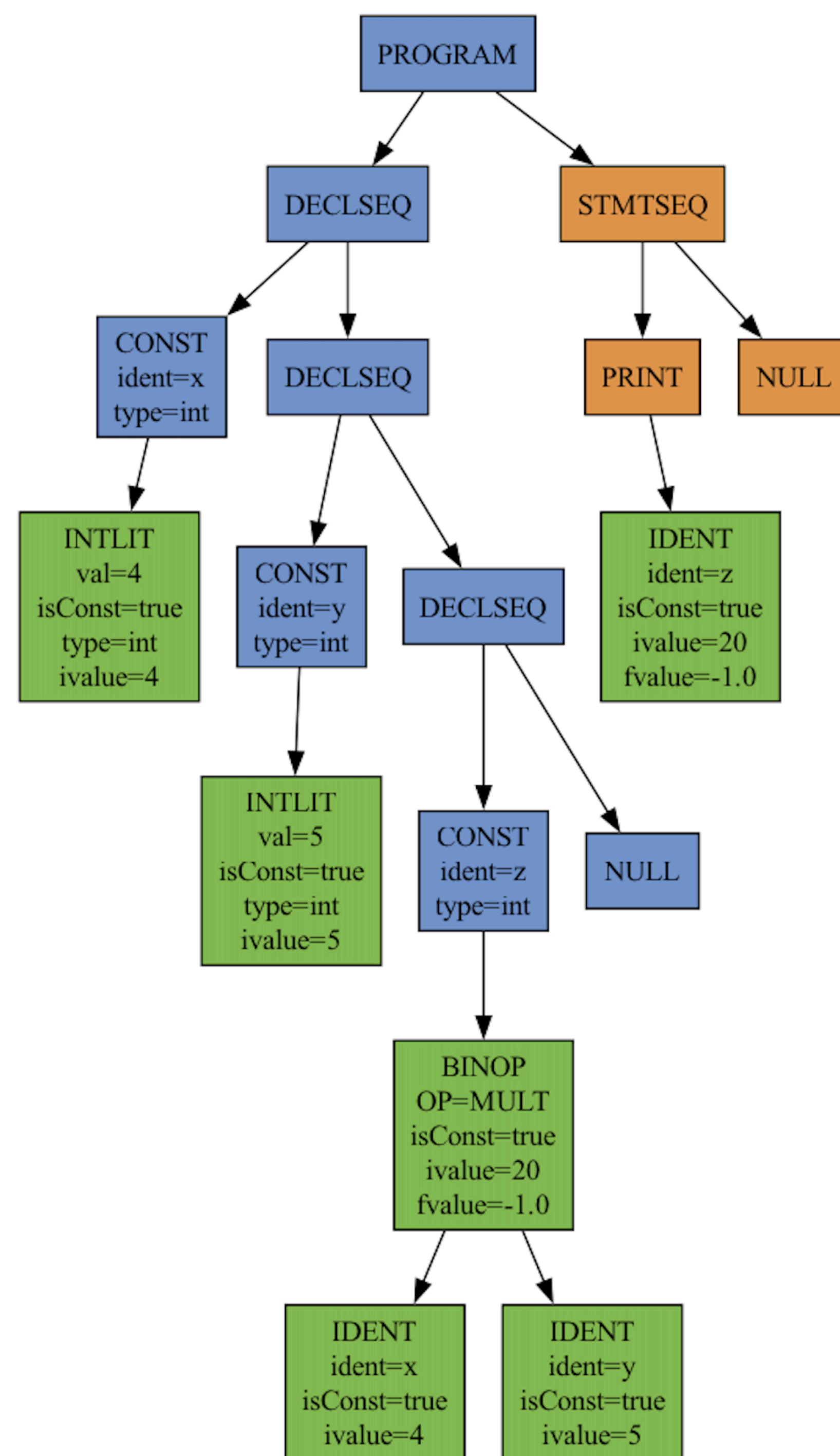
```
> java Sem 4.tny
> dot -Tpng parse_ast.gv > parse_ast.png
> dot -Tpng sem_ast.gv > sem_ast.png
> open parse_ast.png
> open sem_ast.png
```

- Viewing the PNG might require a different command on your operating system.

Visualizing the AST

- On the next page is a visualization of the AST after semantic analysis of this program:

```
BEGIN
    CONST x: int = 4;
    CONST y: int = 5;
    CONST z: int = x * y;
    PRINT z;
END
```



TASK 1

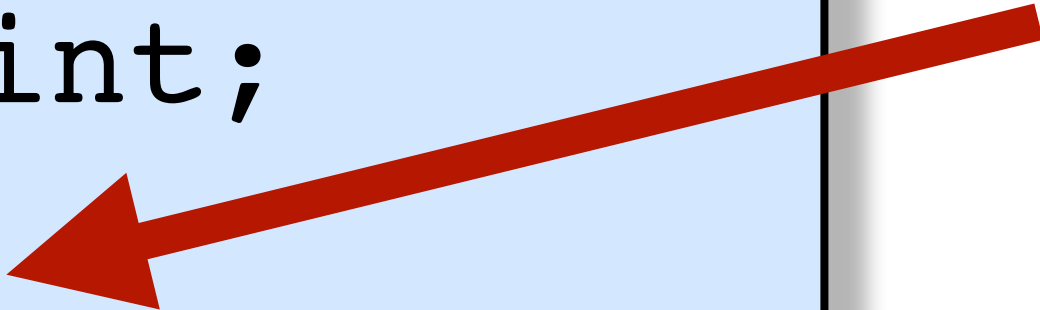
- TYPE CHECKING

Task 1 (a)

- Tiny 2 has int and flt types.
- Mixed arithmetic is not allowed.

```
BEGIN
BEGIN
    CONST x : flt = 1.0;
    VAR    y : int;
    VAR    v : int;
    v = x + y;
END
```

**TYPE
ERROR!**

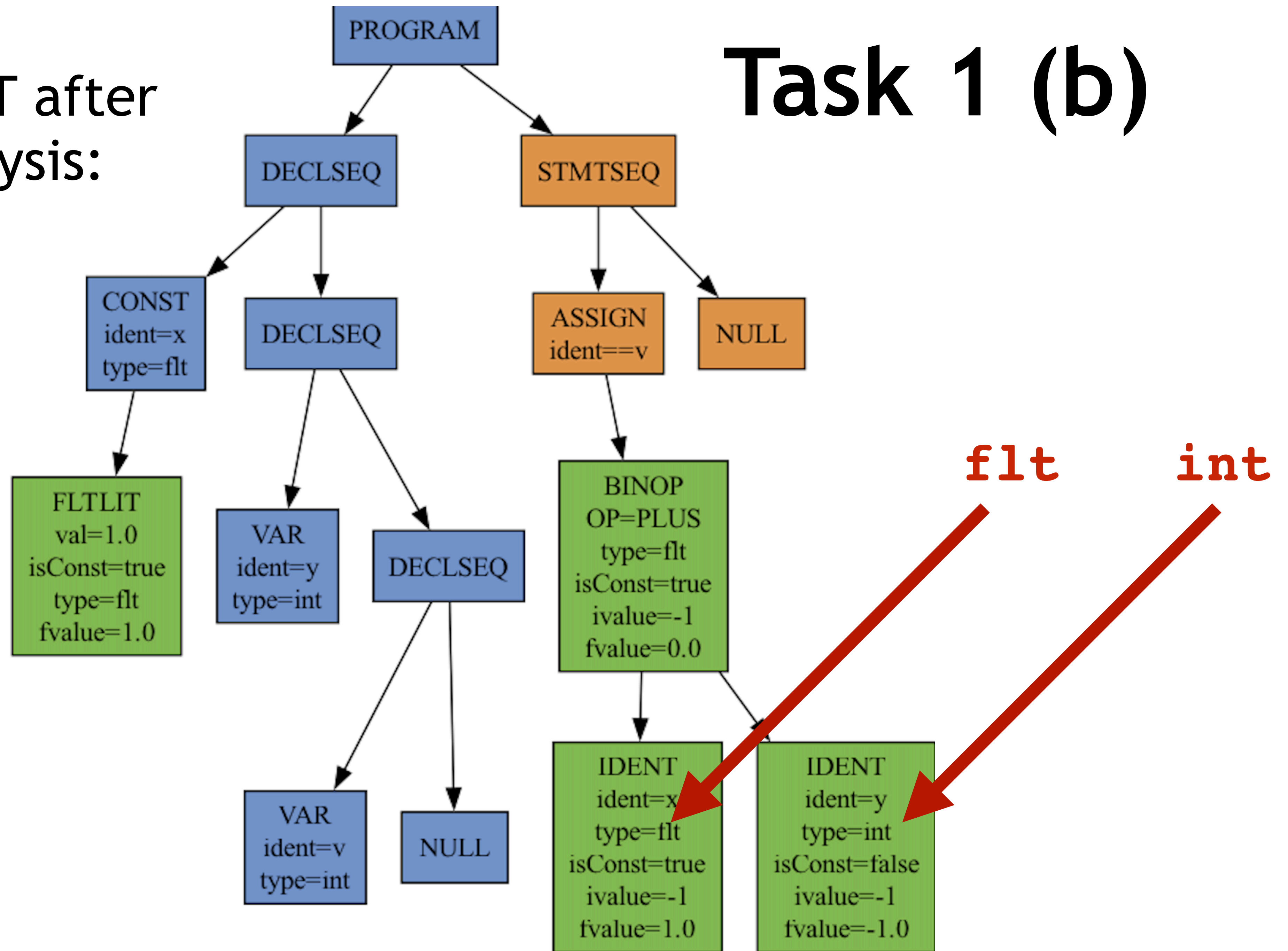


```
> java Sem 7.tny
```

```
[Semantic Error] Wrong binary type: flt; int
```

- Here's the AST after semantic analysis:

Task 1 (b)



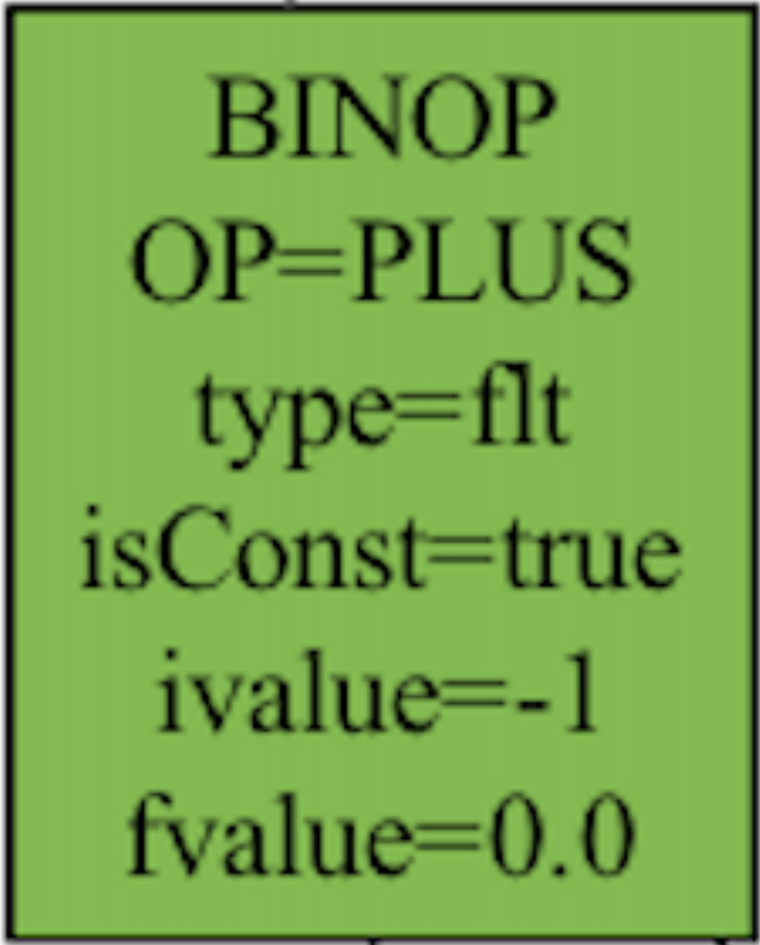
Task 1 (c)

- Your task is to extend `Sem.java` to
 1. compute types for every part of an expression and issue the appropriate error messages
 2. check the types of assignment and print statements.

Task 1 (d)

- To solve this problem I've added a "type" attribute to all expression nodes.
- Since all expression nodes inherit from EXPR we add the attribute there:

```
public abstract class EXPR extends AST {  
    public String type = "";  
}
```



BINOP
OP=PLUS
type=flt
isConst=true
ivalue=-1
fvalue=0.0

Task 1 (e)

- Your task is to search for "TASK_1" in the Sem.java to see the places that require your attention.

```
void constDecl(CONST n) {  
    /* BEGIN TASK_1 */  
    expr(n.expr);  
    ...  
    /* END TASK_1 */  
}  
  
void ident(IDENT n) {  
    /* BEGIN TASK_1 */  
    n.type = ...;  
    /* END TASK_1 */  
}
```

```
void intlit(INTLIT n) {  
    /* BEGIN TASK_1 */  
    n.type = ...;  
    /* END TASK_1 */  
}  
  
void binop(BINOP n) {  
    expr(n.left);  
    expr(n.right);  
    /* BEGIN TASK_1 */  
    n.type = ...;  
    /* END TASK_1 */  
}
```

TASK 2

- CONSTANTS

Task 2 (a)

- The expression in a constant declaration in Tiny 2 must be a constant expression. That is, it cannot contain any variable references:

```
BEGIN
```

```
    CONST x : int = 1;
```

```
    CONST y : int = 1 + 2;
```

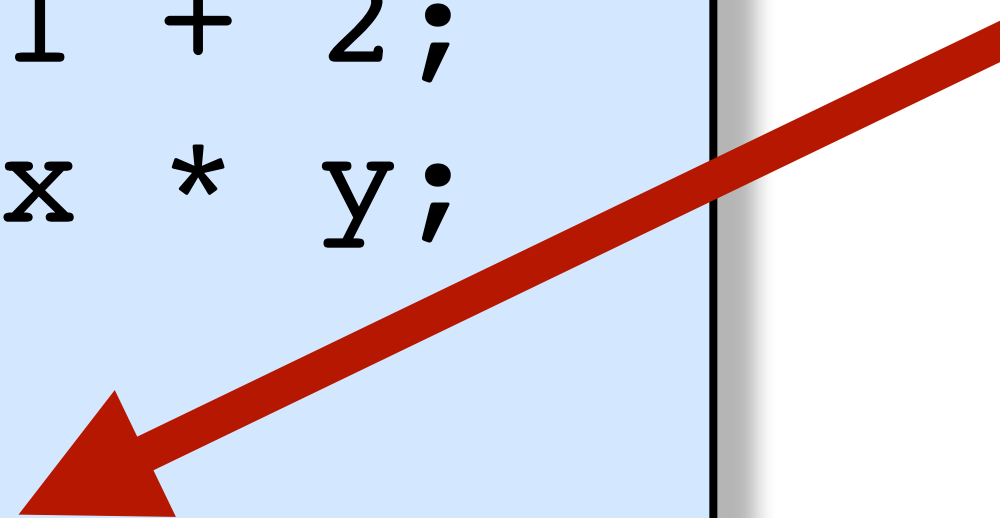
```
    CONST z : int = x * y;
```

```
    VAR    v : int;
```

```
    CONST w = z - v;
```

```
END
```

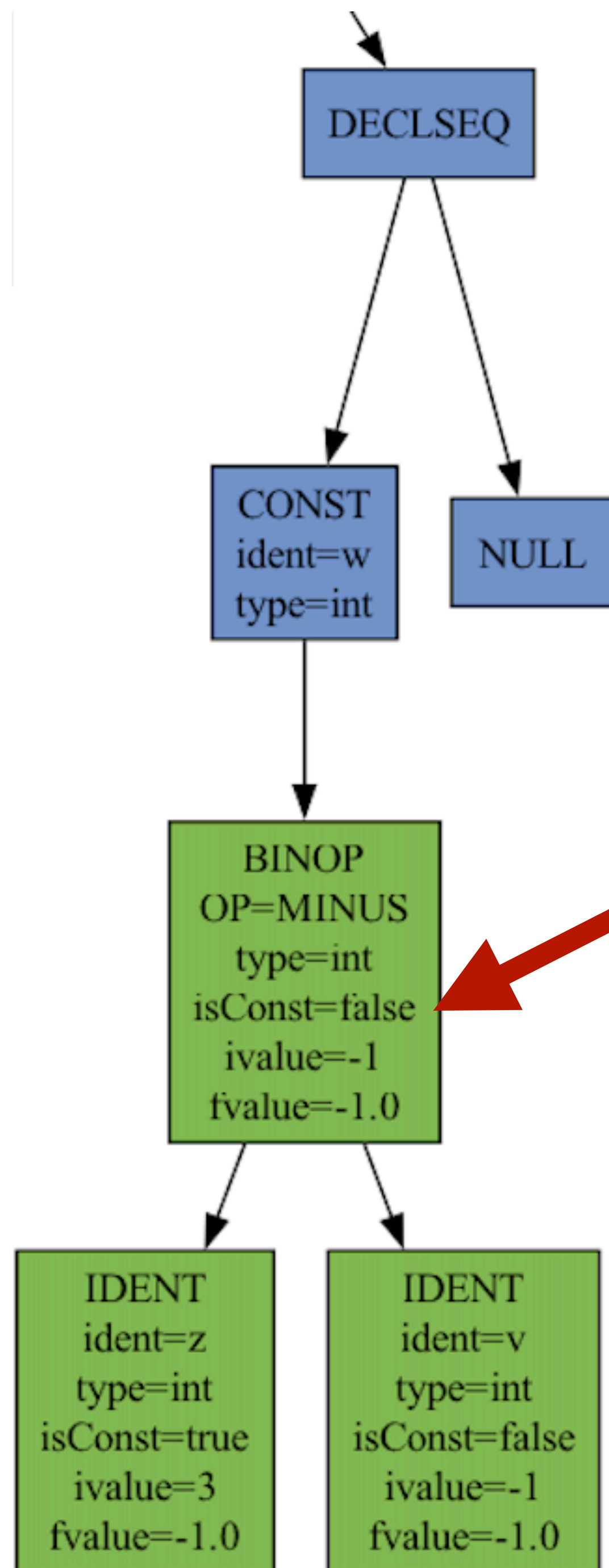
ERROR!



```
> java Sem 5.tny
```

```
[Semantic Error] Constant declaration value not constant: w
```

- Here's a piece of the AST after semantic analysis:



Task 2 (b)

**NOT
CONSTANT !**

Task 2 (c)

- Your task is to extend `Sem.java` to
 1. evaluate constant expressions and
 2. generate an error message if a constant declaration expression isn't a constant value.
- Search for "TASK_2" in `Sem.java` to see the places that require your attention.

Task 2 (d)

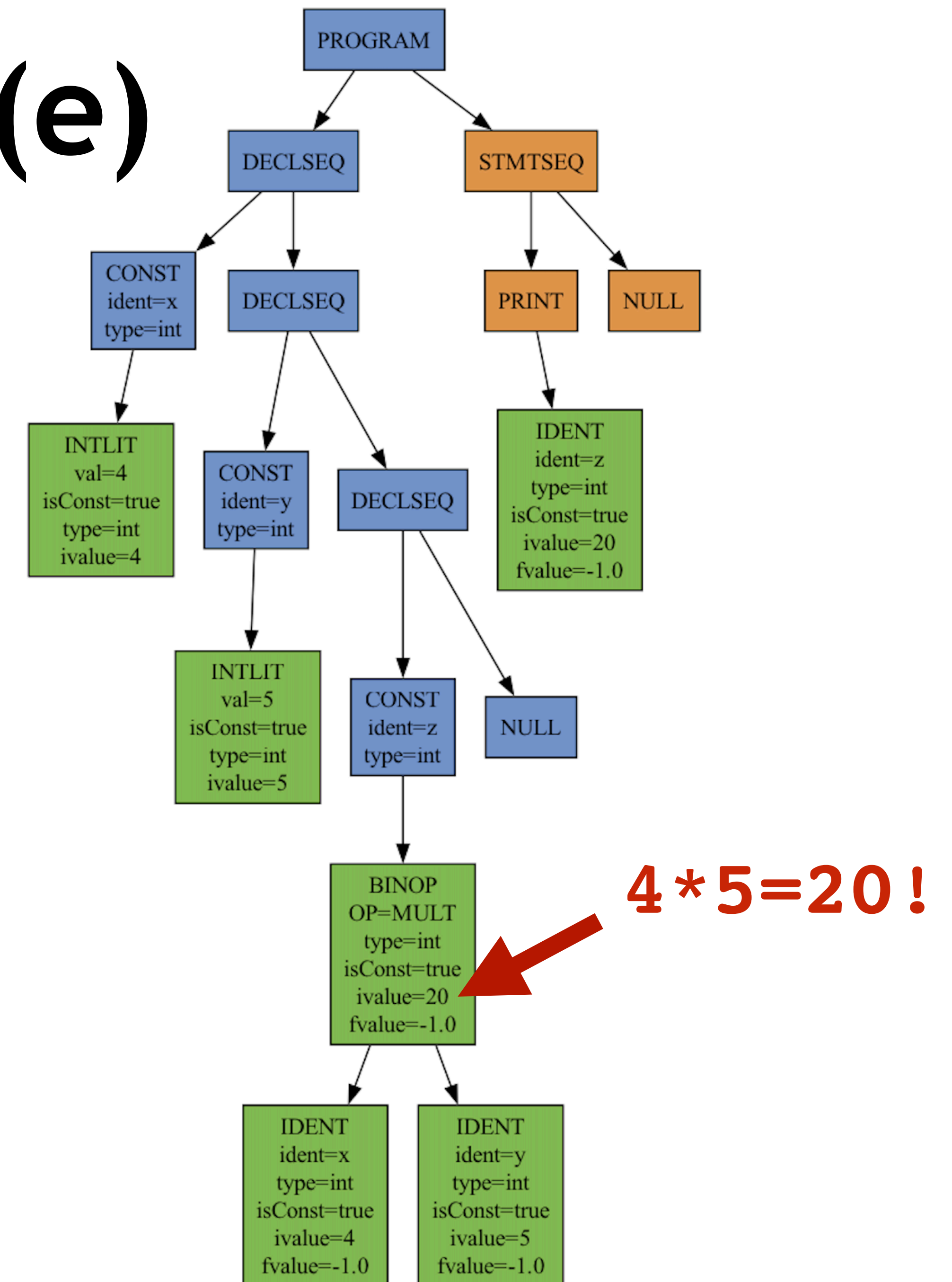
- To solve this problem we need to add 3 attributes to all expression nodes: `isConst`, `ivalue`, `value`.
- `isConst` is true if the expression is, in fact, constant in which case the `value` attribute is set to the evaluated value.
- All expression nodes inherit from `EXPR`, so we add the two attributes there:

```
public abstract class EXPR extends AST {  
    public boolean isConst = false;  
    public int      ivalue  = -1;  
    public float    fvalue  = -1.0f;  
    public String   type    = "";  
}
```

Task 2 (e)

- Here is an example that shows how the semantic analyzer evaluates constant expressions.

```
BEGIN
  CONST x : int = 4;
  CONST y : int = 5;
  CONST z : int = x * y;
  PRINT z;
END
```



TASK 3

- ENVIRONMENTS

Task 3 (a)

- So far, we've made the symbol table a global variable:

```
public class Sem {  
    AST ast;  
    public SyTab sytab = new SyTab();  
  
    void program(PROGRAM n) {  
        decls(n.decls);  
        stats(n.stats);  
    }  
}
```

Task 3 (b)

- This works fine for simple languages, but it's problematic for languages that support nested scopes:

```
int main() {  
    int x = 5;  
    {  
        int x = 10;  
        {  
            int z = 20;  
        }  
    }  
}
```

Task 3 (c)

- So, instead we're going to add an **inherited attribute env**.
- All nodes in the AST will have this attribute.

```
public abstract class AST {  
    public int pos = -1;  
  
    /* BEGIN TASK_3 */  
    public SyTab env = null;  
    /* END TASK_3 */  
}
```

- This `env` attribute will be passed down to *all* the nodes in the AST!

Task 3 (d)

- Your tasks are to:
 1. Remove the global sytab variable
 2. Add a local sytab variable:

```
void program(PROGRAM n) {  
    /* BEGIN TASK_3 */  
    SyTab env = new SyTab();  
    n.decls.env = n.stats.env = env;  
    /* END TASK_3 */  
    decls(n.decls);  
    stats(n.stats);  
}
```

3. Add code to pass the sytab variable down to all the nodes!

```
BEGIN
```

```
  CONST x : flt = 1.0;
```

```
  CONST y : flt = 1.0 + 2.0;
```

```
  CONST z : flt = x * y;
```

```
  VAR    v : flt;
```

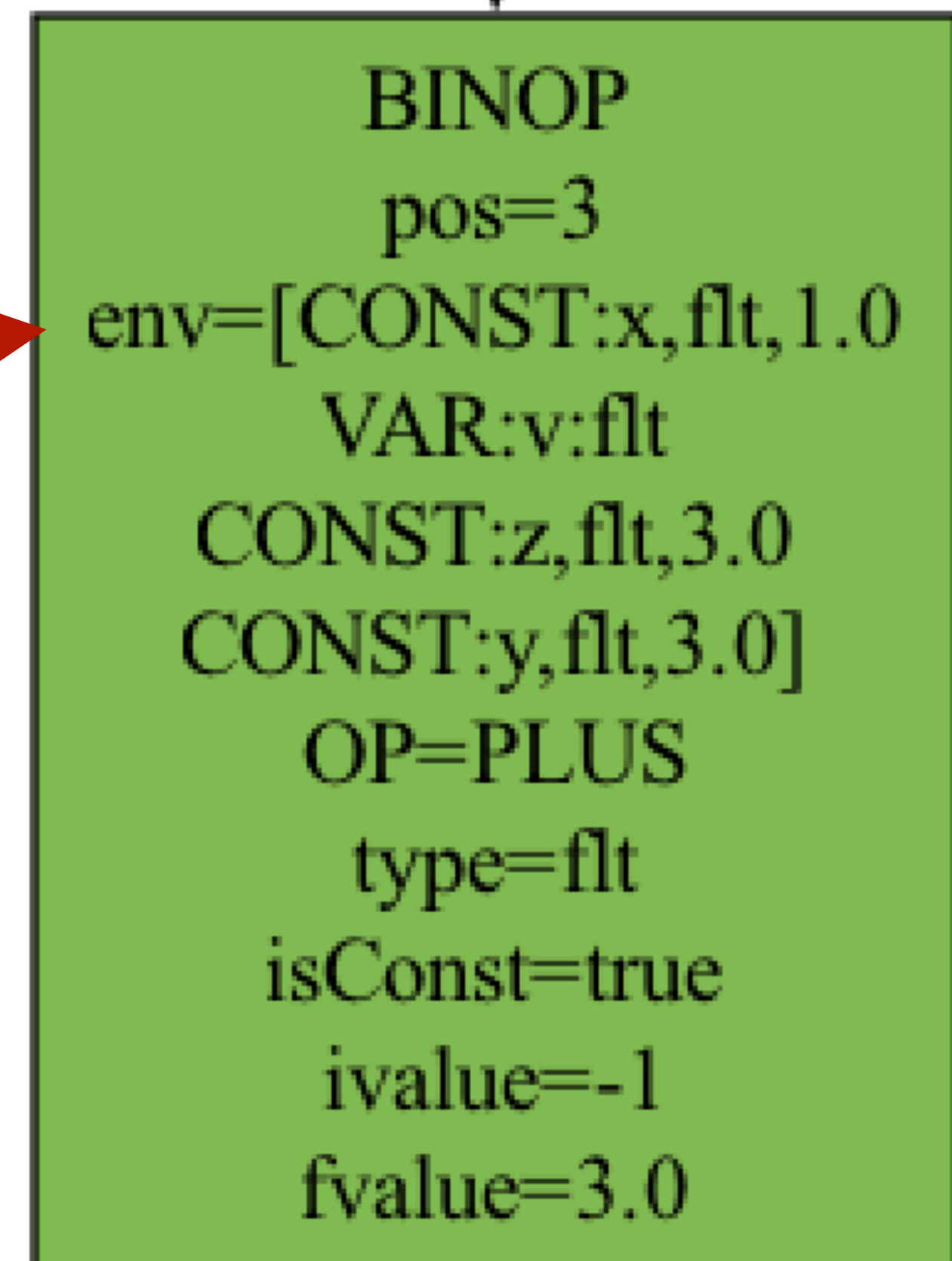
```
  v = z;
```

```
END
```

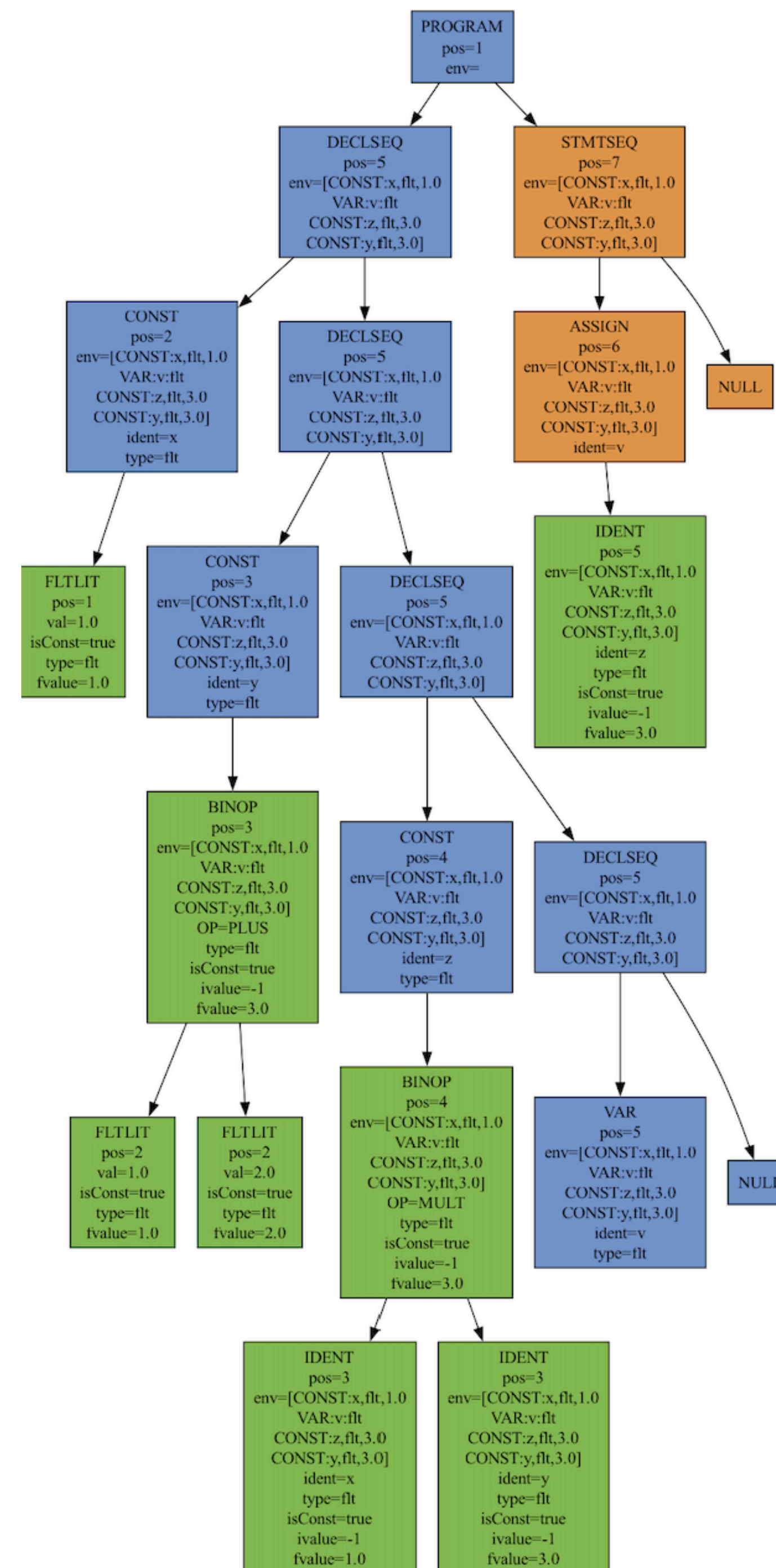
- Here's the node that corresponds to the expression on line 3.

This is the env!

env=[CONST:x,flt,1.0;
 VAR:v:flt;
 CONST:z,flt,3.0;
 CONST:y,flt,3.0
]



- Here's the complete AST with env attributes in every node.



TASK 4

- STANDARD TYPES

Task 4 (a)

- So far, the standard types "int" and "flt" have been hardcoded as strings. This is ugly!
- A real language can have many standard identifiers so we need to treat them better.
- Create a new class `StandardType` that extends `Symbol`, and which will allow us to add "int" and "flt" to the symbol table:

```
public class StandardType extends Symbol {  
  
}
```

Task 4 (b)

- Then :

1. Add `int` and `flt` to the environment variable:

```
void program(PROGRAM n) {  
    SyTab env = new SyTab();  
    /* Add int and flt to env. */  
    n.decls.env = n.stats.env = env;  
    decls(n.decls);  
    stats(n.stats);  
}
```

2. Update all uses of "int" and "flt" to look these up in the environment.



DEFINITIONS AND ALGORITHMS

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA